Gaussian-process reduced models versus neural regression for parametric PDEs: a staged-preregistered benchmark with largely negative confirmatory results

Authors: [AUTHOR NAMES AND AFFILIATIONS — to be supplied by the authors]. **Corresponding author:** [to be supplied].

Abstract

Background. Practitioners choose between linear-subspace (POD-based) and neural surrogates for parametric PDEs with little guidance on when either wins, and comparisons are often confounded by unequal tuning. **Methods.** We compared five surrogates (three POD-regressor and two neural) under a common random-search protocol on 36 synthetic PDE instances, training budgets of 32–1024 solves, three seeds and tie-aware outcomes (±0.10 decade), with controls for the subspace, the kernel prior and hyper-parameter search. Hypotheses, code and success criteria of each stage were committed to version control before the corresponding runs. **Results.** Of 24 instances, the linear-subspace family was clearly better in 13, the neural family in 8, with 3 ties, at N = 1024 (N = 32: 5, 5, 14); the N = 1024 counts are confounded with a doubling of the neural training steps. On smooth, mostly three-parameter problems the advantage reached up to about three decades. Three of the four Stage-1 confirmatory hypotheses were not supported, and a pilot-data diagnostic was not confirmed on new instances (inconclusive; it correlates with parameter dimension). A Gaussian process without POD matched the POD pipeline, and a kernel-ridge control that initially looked 14× worse came within a median 2.7× once its hyper-parameter ranges were widened (a post hoc control prompted by an independent review): the advantage reflects regressor and search quality rather than the subspace, and marginal likelihood per se was not isolated. Training the networks four times longer changed the class of 15 of 48 cells, always toward the neural family. **Conclusions.** In this synthetic benchmark the apparent winner is governed by smoothness, dimension, regressor and search quality, training budget and tuning symmetry; conclusions about ties and learning-curve crossovers are fragile. Results are limited to synthetic problems and lightly tuned networks.

Keywords: surrogate modelling; parametric PDE; reduced-order model; Gaussian process; neural network; benchmark; preregistration

1 Introduction

A parametric partial differential equation (PDE) defines a map from a vector of parameters μ to a solution field. Surrogates of this map are used in design, uncertainty quantification and control, where the full-order model (FOM) is too expensive to call many times. Practitioners typically choose between two families before spending a simulation budget: *linear-subspace* surrogates, in which proper orthogonal decomposition (POD) compresses the snapshots and a regressor maps μ to the reduced coefficients [Hesthaven & Ubbiali 2018; Guo & Hesthaven 2018; Bhattacharya et al. 2021], and *nonlinear-output* neural networks that map μ directly to the field, possibly through a latent bottleneck [Fresca & Manzoni 2022; Franco et al. 2023].

Approximation theory gives a qualitative guide: a small linear subspace suffices when the solution manifold has a rapidly decaying Kolmogorov n-width [Cohen & DeVore 2015], whereas slow decay has been established for the wave equation with discontinuous initial data [Greif & Urban 2019] and is the standard motivation for nonlinear reduced models of transport-dominated problems [Franco et al. 2023]. For operator learning with PCA-Net, slow decay of the PCA eigenvalues has been identified as an obstacle to efficient learning through lower complexity bounds [Lanthaler 2023], which motivates a spectral-decay feature as a diagnostic. It has also been argued that many neural-surrogate benchmarks sit on low-dimensional manifolds where any competent reduced model interpolates well [Duraisamy 2026, preprint]. Kernel and Gaussian-process (GP) methods have been reported to be competitive with DeepONet and the Fourier neural operator [Batlle et al. 2024]; within a POD-regression framework for extruder flow, GP regression was reported to have several advantages over a neural network [Bērziņš et al. 2020, preprint]; and a comparison of neural and polynomial operator surrogates found no universally superior method [Westermann et al. 2026, preprint]. A review of machine-learning solvers for fluid PDEs found that 79% of the articles claiming to beat a standard method used a weak baseline [McGreivy & Hakim 2024].

What remains less clear, and is the subject of this paper, is quantitative and methodological. (i) *How large* are the differences between the families, and in how many problems are they large enough to matter, as opposed to practically tied? (ii) *Why* does a linear-subspace surrogate win where it does: because of the subspace, because of the kernel prior, or because of how its hyper-parameters are fitted? (iii) Can a cheap statistic computed from a pilot set of 32 snapshots predict the winner on problems it has never seen? (iv) How sensitive is the apparent winner to the tuning protocol?

Approach and contributions. We compared five surrogates under a common random-search tuning protocol on 36 simulated parametric-PDE instances, preregistering the hypotheses, analysis code and success criteria of each stage in version control before the corresponding results existed. Contributions: (1) a tie-aware map of where each family wins, with effect sizes, over six training budgets; (2) controls that locate the Gaussian-process advantage in the regressor and its hyper-parameter search rather than in the subspace, including a case where a registered control proved too restrictive; (3) an out-of-sample test of a pilot-data diagnostic, whose frozen form was **not** confirmed; (4) a quantification of how unequal tuning inflates apparent neural-network wins, and of the limits of calling a tuning budget "matched" across method classes; (5) open code. We propose no new surrogate and claim no state-of-the-art accuracy.

2 Related work

Linear-subspace and kernel surrogates. Non-intrusive reduced-order models regress POD coefficients; neural networks were used for this by Hesthaven and Ubbiali [2018] and in PCA-Net [Bhattacharya et al. 2021], and GP regression for reduced models by Guo and Hesthaven [2018]. Bērziņš et al. [2020, preprint] compared radial-basis-function, Gaussian-process and neural-network regression inside a standardised POD framework for extruder flows and reported that GP regression offers several advantages over the neural network, which is the nearest precedent we found for our qualitative finding. Gaussian processes are the standard low-dimensional regressor [Rasmussen & Williams 2006]; a hybrid GP/neural operator-learning framework was proposed by Mora et al. [2025]. Batlle et al. [2024] develop a general kernel framework with a priori error analysis and report that it is competitive with DeepONet and the Fourier neural operator.

Neural operators and nonlinear manifolds. DeepONet [Lu et al. 2021], the Fourier neural operator [Li et al. 2021] and the neural-operator framework [Kovachki et al. 2023] learn maps between function spaces; deep autoencoders learn nonlinear latent manifolds, either inside projection-based reduced models [Lee & Carlberg 2020] or for the parameter-to-solution map [Fresca & Manzoni 2022; Franco et al. 2023]. Our setting has a finite-dimensional parameter, so we study direct and bottleneck networks rather than operator-input architectures. Slow n-width decay for transport and wave problems is established [Greif & Urban 2019].

Fair comparison. de Hoop et al. [2022] study the cost–accuracy trade-off of neural architectures; Lu et al. [2022] compare DeepONet and FNO; Westermann et al. [2026, preprint] compare neural and polynomial surrogates by Pareto frontiers over hyper-parameter ensembles, finding polynomial surrogates more data-efficient for smooth inputs and the FNO reaching the fastest convergence rates for rough ones; Zhang et al. [2026, preprint] define a breakeven complexity that includes data-generation and training costs, while Huang and Stuart [2026, preprint] study post-training many-query cost against classical solvers with data and training costs treated as amortised; McGreivy and Hakim [2024] document weak baselines and reporting bias; Grossmann et al. [2024] compare physics-informed networks with finite elements. Earlier work attempted to derive rules for choosing a surrogate form from data characteristics for design-space approximation [Williams & Cremaschi 2019]. Random search is an efficient alternative to grid search for hyper-parameter optimisation [Bergstra & Bengio 2012], and reporting results as a function of tuning budget and accounting for run-to-run variance have been advocated [Dodge et al. 2019; Bouthillier et al. 2021].

Positioning. The qualitative regime picture, the competitiveness of kernel/GP methods and a preference for GP over neural regression in POD frameworks are established. The present work adds a matched-protocol quantification across the instance suite with equivalence bands, mechanism controls for the Gaussian-process advantage, a pre-registered out-of-sample test of a pilot diagnostic, and a measurement of tuning-protocol sensitivity. *Reading depth and search coverage.* Reading was limited and uneven. Westermann et al. and Zhang et al. were read in large parts of a converted HTML text (we cannot give a reliable percentage); all other sources were assessed from abstracts or bibliographic metadata, a few from titles only. The searches covered the arXiv API, Crossref and web search; one targeted arXiv query for POD+GP versus neural networks returned no hits; Google Scholar, publisher full texts and patent databases were not searched. The absence of a prior pilot-data selection rule is therefore weak evidence.

3 Surrogate families, tuning protocol and diagnostics

Problem. A parametric PDE defines a map $\mu \in \mathbb{R}^p \mapsto u(\mu) \in \mathbb{R}^m$, where u is the FOM solution on an output grid ($m \in \{128, 256\}$). Given N training pairs, a surrogate $\hat{u}$ is scored by the mean relative L_2 error $\mathrm{err} = (1/n)\, \Sigma_j \, \|\hat{u}(\mu_j) - u(\mu_j)\|_2 \,/\, \|u(\mu_j)\|_2$ over $n = 1000$ held-out parameters.

Linear-subspace (LS) family. POD of the centred training snapshots gives $u(\mu) \approx \bar{u} + V_r\, a(\mu)$; the methods differ in the regressor for $a(\mu)$: *POD+GP* (ARD Gaussian processes with Matérn-5/2 or RBF kernel, hyper-parameters fitted by maximising the marginal likelihood on up to 300 points for the leading four modes, the remaining modes reusing the

fourth kernel); *POD+Poly* (ridge-regularised total-degree polynomial, degree ≤ 6, ≤ 5,000 features); *POD+MLP* (multilayer perceptron on the scaled coefficients).

Nonlinear-output (NL) family. *MLP (direct)* maps μ to the full field; *Bottleneck MLP* passes μ through a latent layer of width k ∈ {2, 4, 8, 16} before the field decoder.

Mechanism controls. *POD+KRR*: POD with kernel ridge regression (Matérn-5/2 or RBF, isotropic length-scale, ridge), tuned by the same random search as every other method and **without** marginal-likelihood fitting. *POD+GP-iso*: POD with a GP whose kernel has a single isotropic length-scale, fitted by marginal likelihood. Together with POD+GP (anisotropic, marginal-likelihood fitted), these separate the roles of the kernel prior, anisotropy and marginal-likelihood fitting. *POD+KRR-wide* (Exp H, added after an independent review) is the same kernel ridge with a much wider ridge (10^{-16}–10^{-2}, times N) and length-scale range and a symmetric-eigendecomposition solve, searched with B = 8 or 32 trials. *GP-full* is a GP on the full output field without POD (one shared ARD kernel fitted on at most 300 points; B = 2). Methods are compared in the family-level outcome $\bar{y}$; the mechanism controls compare against the oracle-best network (by test error) in cells where POD+GP-ARD clearly wins.

Tuning protocol. Every method receives exactly B = 8 random-search trials (trial 0 is a fixed default) scored on a random 20% validation split of the N samples; the validation-best configuration is refit on all N samples. Neural networks use Adam with cosine learning-rate decay for 1500 epochs (float32, CPU). Test errors of each trial are logged for the tuning analysis but never used for selection.

Outcome. For each (instance, N, seed), LS* and NL* are the validation-best methods of each family; $\bar{y}$ = mean over seeds of $\log_{10}$(err_NL* / err_LS)*, so $\bar{y}$ > 0 means the LS family is better. A cell is a* tie* if $|\bar{y}| \le 0.10$ decades (an error ratio within about 26%), *LS-clear* if $\bar{y} > 0.10$ and *NL-clear* if $\bar{y} < -0.10$.

Pilot diagnostic. From the first $n_0 = 32$ training snapshots only, f1 = $\log_{10}$ of the mean leave-one-out relative POD reconstruction error with r = 4 modes, and f2 = the slope of $\log_{10} \sigma_k$ against k ($k \le 10$).

4 Experimental design and preregistration

Instances. Thirty-six simulated instances in five sets (Table 1): the *original suite* (16 instances, 8 families × 2 levels), *Experiment A* (8 new instances in 4 new families: high-dimensional Darcy with p = 16 and 32, a Darcy problem with a moving discontinuous inclusion, KdV solitons, and smooth nonlinear conduction), *Experiment A2* (8 further candidates: nonlinear reaction–diffusion, Helmholtz-type, heat with parametrised diffusivity, 1-D diffusion) and *Experiment A3* (4 probes of the diagnostic–dimension confound: two nearly linear Darcy problems with p=16 and 32, and two p=3 problems with moving features). Each solver is verified by unit tests against exact or manufactured solutions (Poisson series, translation, d'Alembert, KdV soliton, Helmholtz and reaction–diffusion manufactured solutions) or grid refinement. **All data are synthetic.**

Data and budgets. For every instance: 4096 i.i.d. training parameters, 1000 interpolation-test and 500 extrapolation-test parameters, from streams seeded by the instance name. Seed s uses the first N elements of a seed-specific permutation, N ∈ {32, 64, 128, 256, 512, 1024} (A2 and the isotropic-GP control: N ∈ {64, 256, 1024}), seeds 0–2.

Staged preregistration (all in version control; commit hashes in the supplement). *Stage 1* (original study): hypotheses H1a, H1b, H2, H4a, analysis code and success criteria committed before any result. *Stage 2*: after a forensic audit of Stage 1, a revised protocol was committed before any Stage-2 comparative result: Experiment A (frozen rules derived from Stage-1 data only; primary tests P1, P2, secondary S1), Experiment B (stronger neural-network refits with explicit sign-retention criteria), Experiment D (kernel-ridge mechanism test with a pre-specified criterion). *Stage 3*: after seeing Experiment A (diagnostic not confirmed) and Experiment D, an addendum fixed, before the runs, Experiment A2 (the 8 candidates; qualifying instances chosen by a pilot-only rule f1 < −2; a prediction and its success criteria) and Experiment E (isotropic-GP control with a pre-specified decision rule). *Stage 4* (addendum v2.2, before the runs): Experiment A3 (descriptive probes of the diagnostic–dimension confound), Experiment G (GP without POD) and Experiment F (float64 and L-BFGS refits of the networks). *Stage 5* (addendum v2.3, after an independent AI review of the draft showed on one cell that the kernel-ridge control's ridge range was too restrictive): Experiment H (wide-range kernel ridge, with predictions and decision rules fixed before the run). Stage-3 experiments are post hoc in *motivation* but preregistered in *prediction*. The 1e-16–1e-2 range of Experiment H and its two budgets (B = 8, 32) were chosen with knowledge of that review cell. The preregistrations are git histories on the authors' repository, not independent registry entries.

Statistics. Stage-1 confirmatory tests (H1a, H1b, H2, H4a) used Holm correction; Experiment A tests P1 and P2 used Holm correction; all other analyses are labelled exploratory. The unit of inference is the instance (or family), not the (instance, N) cell.

Table 1 / per-instance outcomes at N = 1024

Instance	p	f1	y1024	Class	Set
bratu_L1	3	-3.75	3.31	LS-clear	original
bratu_L2	3	-3.27	3.08	LS-clear	original
helm_L1	3	-3.94	2.93	LS-clear	A2
heatp_L2	3	-3.37	2.9	LS-clear	A2
heatp_L1	3	-3.79	2.87	LS-clear	A2
nlheat_L1	3	-3.91	2.72	LS-clear	Exp A
diff1d_L1	4	-2.18	2.7	LS-clear	A2
helm_L2	3	-4.16	2.64	LS-clear	A2
nlheat_L2	3	-3.45	2.58	LS-clear	Exp A
reacdiff_L1	3	-4.13	2.57	LS-clear	A2
reacdiff_L2	3	-3.75	2.38	LS-clear	A2
burgers_L1	4	-2	2.25	LS-clear	original
heat2d_L1	4	-1.15	1.38	LS-clear	original
darcy_L1	4	-1.16	1.13	LS-clear	original
diff1d_L2	8	-1.72	1	LS-clear	A2
wave_L1	4	-0.62	0.78	LS-clear	original
transport_L1	4	-0.88	0.75	LS-clear	original
fisher_L1	4	-1.81	0.52	LS-clear	original
darcylin_L1	16	-1.57	0.48	LS-clear	A3
kdv_L2	4	-1.03	0.29	LS-clear	Exp A
advdiff_L1	4	-0.78	0.21	LS-clear	original
darcylin_L2	32	-1.6	0.18	LS-clear	A3
burgers_L2	4	-0.65	0.11	LS-clear	original
wave_L2	4	-0.27	-0.04	tie	original
darcyhd_L2	32	-0.49	-0.07	tie	Exp A
advdiff_L2	4	-0.59	-0.07	tie	original
darcyhd_L1	16	-0.49	-0.13	NL-clear	Exp A
incl_L2	4	-1.29	-0.13	NL-clear	Exp A
darcy_L2	10	-0.36	-0.19	NL-clear	original
incl3_L1	3	-1.29	-0.21	NL-clear	A3
kdv_L1	4	-0.57	-0.23	NL-clear	Exp A
incl_L1	4	-1.69	-0.25	NL-clear	Exp A
transport_L2	4	-0.42	-0.33	NL-clear	original
heat2d_L2	4	-0.22	-0.34	NL-clear	original
fisher_L2	4	-1.61	-0.44	NL-clear	original
heatmv3_L1	3	-0.23	-0.44	NL-clear	A3

Table 1. Instances ordered by $\bar{y}_{1024}$ (> 0: linear-subspace better). f1 is the pilot diagnostic; p the parameter dimension. Sets: original (16), Exp A (8, new families, independent test of a frozen rule), A2 (8 smooth candidates), A3 (4 probes).

5 Results

All numbers below are produced by `manuscript/build_v3.py` from the result files in the repository; verdicts are against criteria registered before the corresponding runs (§4). Unless stated, $\bar{y}$ is the seed-mean of $\log_{10}$(err_NL/*err_LS*) with validation-selected winners, and the unit of inference is the instance. Two cautions apply to every count below: (i) **all N = 1024 results are confounded with a change in the neural training schedule** — full-batch Adam (1500 steps) for N ≤ 512 but mini-batches of 512 (3000 steps) at N = 1024, so the networks receive twice the optimisation steps there (their median error gain per doubling of N is 0.10 decade from 256 to 512 but 0.24 from 512 to 1024, against 0.23 and 0.22 for the linear-subspace family); (ii) the three random-search draws per method (one per seed) are **shared across all instances and all N** (design property, disclosed in the deviations log), so seeds are not independent replicates of the tuning.

5.1 Where each family wins

Across the 24 instances of the original study and Experiment A, the outcome depends strongly on the training budget (Figure 1, Table 2). At N = 32, 5 instances favour the linear-subspace family clearly, 5 the neural family and 14 are ties; at N = 1024 the counts are 13, 8 and 3 (the NN-clear count is 6–8 depending on the tie band, Table 5). Where the linear-subspace family wins it can win by orders of magnitude: $\bar{y}_{1024}$ = 3.3 (bratu_L1), 2.7 (nlheat_L1), 2.2 (burgers_L1); where the neural family wins, the margin is at most 0.44 decade at the default budget. All 8 smooth candidates of Experiment A2 are linear-subspace-clear at N = 1024 ($\bar{y}$ between 1.0 and 2.9). The neural-favourable cases are concentrated in problems with moving or discontinuous features (transport_L2, heat2d_L2, incl, kdv_L1) and in some high-dimensional parameter spaces (darcyhd, p = 16 and 32); **however, a Gaussian process on the full field without POD (Exp G) beats the best network in several of these** (Table 4), so part of the neural advantage in this map is attributable to POD truncation of the *particular* pipeline rather than to neural regression. The earlier headline that neural networks "win most cells at N = 32" is not supported: most of those cells are ties (Figure 1).

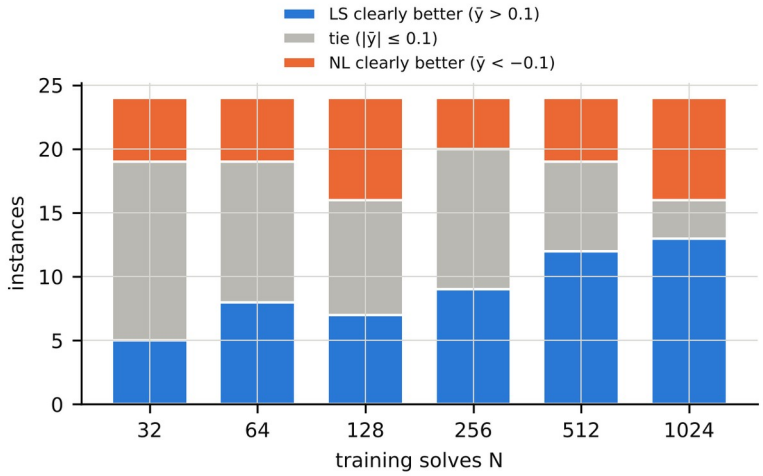


Figure 1. Number of instances (out of 24: 16 original + 8 Exp A) in which the linear-subspace family is clearly better, tied, or clearly worse than the best neural network at each training budget (tie band ±0.10 decade).

N	LS-clear	tie	NL-clear
32	5	14	5
64	8	11	5
128	7	9	8
256	9	11	4
512	12	7	5
1024	13	3	8

Table 2. Cell classification of the 24 instances by training budget (tie band ±0.10 decade). N = 1024 is step-count-confounded (see above).

Stage-1 confirmatory hypotheses were largely not supported (Table 3): the family difference at N = 32 has median $\bar{y}$ = -0.059, and $\bar{y}$ did not fall systematically with N (5 of 16 instances). Because of the step-count change we make no claim about learning-curve crossovers.

ID	Hypothesis / criterion	Observed	raw p	Holm-adj. p	Verdict
H1a (Stage 1)	median $\bar{y}$ at N=32 > 0	median -0.059	0.874	1.000	not supported
H1b (Stage 1)	$\bar{y}$ falls with N in most instances	5/16 with ρ<0	0.962	1.000	not supported
H2 (Stage 1)	pilot features beat {$\log_2$N, p} (leave-one-family-out)	ΔRMSE 3.25, CI [-0.07, 6.40]	0.048	0.145	not supported
H4a (Stage 1)	asymmetric tuning inflates NL wins	10 vs 1 discordant cells; NL-win share 47%−56%	0.006	0.023	supported at cell level; see text
P1 (Exp A)	Spearman(f1, $\bar{y}_{1024}$) < 0, 8 new instances	ρ = -0.381	0.18	0.180	not rejected

ID	Hypothesis / criterion	Observed	raw p	Holm-adj. p	Verdict
P2 (Exp A)	frozen R2 beats frozen B0 out of sample	gain 0.598, CI [-0.001, 1.33] (10^6 draws)	0.025	0.051	not rejected (knife-edge)
S1 (Exp A)	frozen threshold rule beats majority (70.8%)	6/10 correct	0.864	–	not supported

Table 3. Pre-registered confirmatory and primary tests. Holm adjustment within each stage. H2 compares against the baseline {log₂N, p}, whose leave-one-family-out RMSE (4.17) is far worse than that of {log₂N} alone (1.12) because p = 10 is extrapolated from p ≤ 4; against {log₂N} the gain is about 0.2 and the verdict is unchanged. "Knife-edge": the lower CI limit is −0.001. S1: in the 10 new non-tie cells the majority share is 0.50, 4 of the 6 correct cells are nlheat, so the test is uninformative.

5.2 A pilot diagnostic: inconclusive

A frozen rule built from the original data (threshold t = −0.52 on f1; a two-feature regression R2) was tested out of sample on 8 new instances (4 new families, two levels each). P1 was not significant (ρ = -0.381, exact p = 0.180) and P2 was a near miss (out-of-sample squared-error gain 0.598; instance-bootstrap p = 0.025 with 10^6 draws, Holm-adjusted 0.051; the pre-registered 10^4-draw analysis gave 0.027 and 0.055); the frozen threshold rule classified 6 of 10 non-tie cells correctly. The pre-registered verdict is *not replicated*; the right reading is *inconclusive*: the test has 8 instances in 4 families, 88% of the P2 gain comes from the two nlheat instances, and the correlation changes sign without them (ρ = 0.49). In addition, f1 is correlated with the parameter dimension (Spearman ρ = 0.71 across the 24 instances), and among those 24 every instance with f1 < −2 has p = 3 (Figure 2; across all 36 instances diff1d_L1 has f1 = −2.18 with p = 4).

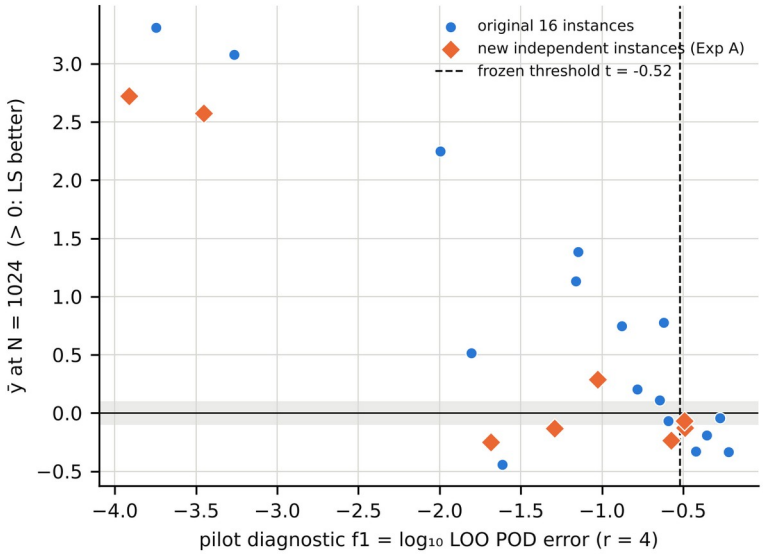


Figure 2. Pilot diagnostic f1 against the outcome ȳ at N=1024 for the 16 original (blue) and 8 independent Exp A (orange) instances; the dashed line is the threshold frozen from the original data. Instances with f1<−2 all have p=3.

Two follow-up experiments probed this confound (neither can be confirmatory). In Experiment A2, a pilot-only rule (f1 < −2) selected 7 of 8 candidate instances, which belong to four authors-designed smooth families (two levels each); all 7 reached $\bar{y}_{1024} \geq 1$ decade (the pre-registered criterion was ≥ 4 of 7 against a baseline rate of 0.15, which was met; we do not attach weight to the resulting binomial p-value because the 7 instances are not independent and the candidates were chosen after smooth p = 3 problems were seen to be won by the GP). Six of the seven have p = 3, and the single control (p = 8, f1 = −1.72) had $\bar{y}_{1024}$ = 0.997, practically the same as the "decisive" ones, so A2 does not show that f1 separates anything beyond smooth low-dimensional problems. In Experiment A3, two p = 3 problems with moving features (f1 = -0.23, -1.29) favoured the neural family ($\bar{y}_{1024}$ = -0.44, -0.21), consistent with f1 carrying information beyond dimension, while two nearly linear high-dimensional problems (p = 16, 32) had f1 ≈ −1.6 by construction (r = 4 modes cannot capture a nearly linear map in 16 or 32 parameters) and only mildly favoured the linear-subspace family ($\bar{y}_{1024}$ = 0.48, 0.18). Pooled over the 24 instances and leaving one family out at a time, adding f1 to {log₂N} lowers the cross-validated RMSE from 1.07 to 0.62 (exploratory; the features were first selected on 16 of these instances; the baseline {log₂N, log₂p} gives 1.47). We conclude that a cheap pilot statistic is informative about

where the linear-subspace family wins by a lot (smooth, low-dimensional maps), but we cannot show that it is more than a smoothness-and-dimension indicator.

5.3 What drives the Gaussian-process advantage

We ran four controls (Table 4, Figure 3). (i) *Kernel ridge with the registered search* (POD+KRR, ridge λN with $\lambda \in [10^{-10}, 10^{-2}]$, one length-scale, B = 8) is a median 14.3× worse than POD+GP in the 52 cells where POD+GP clearly beats the best network (only 13% within 3×), so the pre-registered conjecture that matched tuning would close the gap (D1) is rejected for *that* control. (ii) *A post-hoc control prompted by an independent review* (Exp H, pre-registered before it ran) widened the kernel-ridge ridge range to $[10^{-16}, 10^{-2}]$ and the length-scale range to [0.1, 30]: the median ratio to POD+GP in the 28 GP-clear cells of the $N \in \{64, 256, 1024\}$ subset falls to 6.5× with B = 8 and to 2.7× with B = 32 (57% of cells within 3×; neither pre-registered attribution, "range" or "marginal likelihood", was met), and the wide-range KRR beats the best network by more than 0.1 decade in 64% of those cells. The Exp D gap was therefore largely a property of the restricted range and coarse search of the control. (iii) An isotropic-kernel GP fitted by marginal likelihood is within a median 1.92× of the anisotropic GP (but more than 3× worse in 29% of cells) and the restricted kernel ridge is 8.1× worse than it; this rule (M-MLII, median-based) was met in these p = 3–4 cells but, given (ii), it cannot be read as isolating marginal likelihood. (iv) A GP on the *full* field without POD matches POD+GP in the GP-clear cells (median ratio 0.95×; G1 supported) and is clearly better than the best network in 18 of 24 instances at N = 1024 against 12 for POD+GP (Table 4). Further, POD+Poly, which has neither a kernel nor marginal likelihood, is a median 9.9× worse than POD+GP in the same 52 cells but still beats the best network by more than 0.1 decade in 23 of them, and a *direct* network beats the network on POD coefficients by more than 0.1 decade in 17 of 24 instances at N = 1024 (the opposite in 1). Our reading is that, in this benchmark, the advantage of the Gaussian-process pipeline is the advantage of a **smooth regressor whose hyper-parameters (notably the effective ridge/noise level) are searched over a wide range and finely**, and of the lightly tuned networks' failure to reach the near-interpolation regime, not a property of the POD subspace; marginal likelihood *per se* was not isolated (the GP's continuous optimisation was not matched in cost by the 32-draw kernel-ridge search).

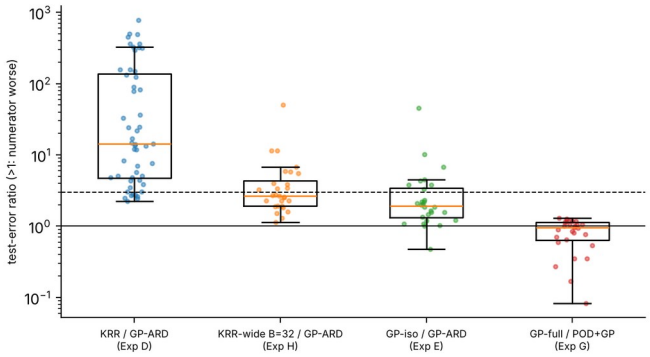


Figure 3. Mechanism controls in cells where POD+GP-ARD clearly beats the best neural network. Dashed line: 3× (the pre-registered tolerance). From left: kernel ridge with the registered restricted ranges (B=8); kernel ridge with wide ranges (B=32, Exp H); isotropic-kernel GP; GP on the full field without POD.

Experiment	Quantity	Observed	Pre-registered verdict
D (KRR control)	median err(POD+KRR)/err(POD+GP) in GP-clear cells	14.3× (share ≤3×: 13%; n=52)	D1 (≤3×, ≥75%) **rejected**
E (iso-GP control)	median err(GP-iso)/err(GP-ARD); err(KRR)/err(GP-iso)	1.92×; 8.1× (n=28)	M-MLII (rule met; low-p cells only; not an isolation of marginal likelihood, see Exp H)
H8 (wide-range KRR, B=8; post hoc)	median err(KRR-wide)/err(POD+GP-ARD), GP-clear cells	6.5× (q25–q75 3.6–14.8; ≤3×: 21%; n=28)	mixed (neither H-RANGE nor H-MLII)
H32 (wide-range KRR, B=32; post hoc)	same	2.7× (q25–q75 1.9–4.4; ≤3×: 57%)	mixed (median ≤3 but only 57% of cells ≤3)
G (no-POD GP)	median err(GP-full)/err(POD+GP) in GP-clear cells	0.95× (q25–q75 0.64–1.14; all ≤3×)	G1 **supported**

Table 4a. Mechanism experiments. All are post hoc in motivation; criteria were registered before each run. Cells are pooled (instance, N) cells selected by an oracle comparator (best network by test error), and are not independent (28–52 cells from 13–14 instances).

N	POD+GP: GP-clear	POD+GP: NN-clear	GP on full field: GP-clear	GP on full field: NN-clear
64	7	13	9	7
256	9	8	15	6
1024	12	9	18	5

Table 4b. Exp G, all 72 cells (24 instances × N ∈{64, 256, 1024}): clear wins (tie band 0.10) of each GP variant against the best network (oracle comparator).

5.4 Robustness, and the limits of "matched tuning"

Check	Result
Tie band 0.05 / 0.10 / 0.15 / 0.30, N=1024 (LS-clear / NL-clear / tie)	13/10/1 ; 13/8/3 ; 12/6/6 ; 10/3/11
Tie band, N=32 (same order)	5/10/9 ; 5/5/14 ; 4/4/16 ; 2/0/22
Seed sign-consistency of clear cells	mean share of seeds with the cell's sign 95.5%; all 3 seeds agree in 87% of 89 clear cells
Config-draw halves (even vs odd random-search trials)	class agreement 83% over 144 cells; sign agreement 99% when both halves clear; Pearson r($\bar{y}$) = 0.988
Metric: median or 95th-percentile error instead of mean	class agreement 95% / 91%; sign flips among 89 clear cells: 0 / 0
Exp B: 4× epochs (6000 vs 1500), N∈{256,1024}	B1 88% (≥85%) pass; B2 median shift 0.24 (<0.5) pass; but 15/48 cell classes change

Table 5. Robustness checks. Rows 1–6 are analysis-only; the Exp B row refits stored configurations (Exp B classes use the best of each family by test error, which differs from validation-selected classes by at most 0.24 decade).

Three results deserve emphasis. First, class labels are only moderately reproducible under a different random-search draw: even and odd search trials (4 trials each; the even half contains the default trial) give the same class in 83% of cells, the sign of clear cells agreeing in 99%; given that draws are shared across instances (above), this under-states the uncertainty of the tuning procedure. Conclusions about individual ties are fragile, those about large gaps much less so. Second, **the neural error floor is partly a training-budget effect**. Training four times longer (Exp B) satisfies both pre-registered criteria (sign retained in 88% of clear cells; median shift toward the neural family 0.24 decade), but it moves $\bar{y}$ toward the neural family in every one of the 15 cells whose class changes (of 48; nine ties become neural-clear) and improves 90% of the refits. **Exp F (float64 + L-BFGS; pre-registered artefact test).** At N=1024 in the 7 instances where the linear-subspace family led by ≥1 decade, the best of the 1500-epoch float32, 6000-step float64 Adam and float64 Adam+L-BFGS refits lowers the neural error floor by a median 0.47 decade; median $\bar{y}$ moves from 2.51 to 2.04; 5 of 7 instances keep $\bar{y} \geq 1$. Pre-registered verdict: **ROBUST** — i.e. the large gaps were not overturned by the tested refits (float64, 6000 full-batch Adam steps, L-BFGS capped at 400 iterations, which changed the training loss by <1% in 21% of refits), not that they are independent of the optimiser. The instances that fall below one decade are darcy_L1, heat2d_L1, none reverses to a neural win. The oracle-best variant is chosen using test error, which favours the neural family by design. The smallest neural error reached falls from 6.8e-04 to 2.0e-04. Third, the neural error at N = 1024 sits at about $5{\cdot}10^{-4}$–10^{-3} in 11 of the 12 instances with $\bar{y} > 2$ irrespective of the problem, which points to a training limit rather than to the approximability of the map; Exp F reduced it only to about $2{\cdot}10^{-4}$.

5.5 Sensitivity to tuning symmetry

In the Stage-1 test of unequal tuning, giving the neural family 8 trials and the linear-subspace family 1 changed the winner in 10 cells in the direction of the neural family and in 1 in the other (exact one-sided binomial/McNemar p = 0.006; neural-win share 47% → 56%). This is a cell-level test on 11 discordant cells out of 96; at the instance level only 5 of 16 instances contribute any flip (Wilcoxon p = 0.031). We read it as direction-consistent evidence that *tuning the neural family more than the linear-subspace family* moves the comparison toward the neural family; the converse asymmetry was not tested. "B = 8 trials" counts trials, not coverage of the search spaces: the POD+GP search has only 8 discrete points and its marginal-likelihood optimisation is not counted in B, whereas the neural search is continuous in several dimensions, and Exp H shows that the outcome for kernel ridge depends strongly on the range and resolution of the search. The linear-subspace family also selects among three methods and the neural family among two.

6 Discussion

What the evidence supports. (i) In this suite of 36 synthetic instances, a Gaussian-process reduced model is far more accurate than float32 networks trained for 1500 epochs on smooth, low-dimensional maps at 256–1024 training solves, and this was reproduced on 7 of 7 pre-selected smooth instances (which are, however, from four related families and mostly p = 3). (ii) The advantage is not due to the POD basis (a no-POD GP matches or beats it) and is largely a hyper-parameter-search effect: a kernel regressor with a wide, finely searched ridge range closes most of the gap, whereas the registered, narrower control did not. (iii) Where networks win against the POD+GP pipeline (moving

or discontinuous features, some high-p problems), a no-POD GP often wins again, so those cells are not clean evidence for neural superiority either. (iv) The class of a cell is sensitive to the neural training budget and, for ties, to the random-search draw.

What it does not support. A pilot-data statistic that predicts the winner on unseen instances (inconclusive at 8 instances); learning-curve crossovers (step-count confound); an isolation of marginal likelihood as the mechanism; any statement about real-world problems, operator-learning architectures (FNO, DeepONet), well-tuned deep networks, or problems with $p \gtrsim 10$, where the mechanism controls are uninformative.

Relation to prior work. The competitiveness of kernel/GP methods [Batlle et al. 2024], the advantage of GP regression over a neural network in a POD framework [Bērziņš et al. 2020] and the absence of a universal winner [Westermann et al. 2026] are known. Our contribution is a quantified, tie-aware map under a common protocol, controls that locate the advantage in the regressor and its hyper-parameter search rather than in the subspace, and an explicit report of how far the answer moves with tuning symmetry, training budget and search draw, including a case in which a registered control turned out to be too weak. We do not claim to be the first to observe that a GP beats a network on smooth problems.

7 Limitations

Synthetic problems only; 36 instances in 19 family labels, several of them small (1-D/2-D, 128–256 outputs), solved by in-house solvers verified by manufactured solutions but not by third-party codes. *Few methods*: five main methods plus controls; no operator-learning networks, no ensembles, no second-order optimisers in the main protocol. *Neural networks are lightly tuned*: B = 8 random draws shared across instances, Adam for 1500 epochs, float32, a step-count change at N = 1024; Exp B and Exp F lower the neural error by 0.25–0.7 decade and move outcomes toward the neural family, so a thoroughly tuned neural pipeline could narrow further; Exp F's L-BFGS arm was capped at 400 iterations (it changed the training loss by < 1% in 21% of refits) and selected the best variant by test error. *Tuning is "matched" only in the number of trials*, with unequal numbers of members (3 vs 2). *The mechanism controls* use oracle-selected, pooled, non-independent cells and a kernel-ridge control that was initially too restrictive. *The diagnostic is confounded with parameter dimension and carried by a few smooth instances. Staged preregistration is self-hosted* (git history on the authors' repository; commit hashes recorded; not an independent registry), several experiments were designed after seeing earlier results (post hoc in motivation), and the original run had disclosed deviations (a non-deterministic seeding of the random search, fixed and replay-verified; an interim look at two of three seeds). *Statistical units are few*: 24 instances in 12 families for the main comparisons; cell-level tests are not independent. The independent "audits" were performed by AI agents, not by human reviewers.

8 Conclusions

On 36 synthetic parametric-PDE instances, Gaussian-process reduced surrogates beat lightly tuned neural networks by up to about three orders of magnitude on smooth, low-dimensional maps; the advantage does not require the POD subspace and shrinks to a factor of about 2–3 when kernel ridge is given a wide, finely searched hyper-parameter range, so it reflects search and regression quality rather than a neural-versus-linear distinction. Networks win on some problems with moving features and some high-dimensional cases, though a no-POD GP often recovers those. A pilot-data diagnostic was not confirmed on new instances. Practitioners comparing surrogate families should report tie-aware outcomes, state the tuning symmetry, search ranges and training budget, and include a well-tuned GP/kernel baseline.

Declarations

Author contributions, affiliations, funding, competing interests: [to be supplied by the authors]. *Use of AI tools:* the study design, code, analyses and this manuscript were prepared with an AI system (Claude) acting as research assistant under the authors' direction; the "independent" audits in the supplement were performed by fresh-context AI agents, not by human reviewers, and the authors must verify every number, citation and claim before submission and take responsibility for it. *Data and code availability:* all data are generated by the code in the repository (`Reinforced_Research_Project/code`) from the instance names and seeds; raw outputs are in `results/`; a permanent archive (e.g. Zenodo DOI) has not been created yet. *Preregistration:* staged, in git history (commit hashes in `research_design/`), not in an independent registry.

References

[1] Batlle P, Darcy M, Hosseini B, Owhadi H (2024) Kernel methods are competitive for operator learning. J Comput Phys 496:112549. 10.1016/j.jcp.2023.112549 / arXiv:2304.13202

[2] Bergstra J, Bengio Y (2012) Random search for hyper-parameter optimization. J Mach Learn Res 13(10):281-305. JMLR 13:281-305

[3] Bhattacharya K, Hosseini B, Kovachki NB, Stuart AM (2021) Model reduction and neural networks for parametric PDEs. SMAI J Comput Math 7:121-157. 10.5802/smai-jcm.74

[4] Bouthillier X, Delaunay P, Bronzi M, et al. (2021) Accounting for variance in machine learning benchmarks. MLSys 2021. arXiv:2103.03098

[5] Bērziņš A, Helmig J, Key F, Elgeti S (2020) Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems. arXiv:2006.13706.

[6] Cohen A, DeVore R (2015) Approximation of high-dimensional parametric PDEs. Acta Numerica 24:1-159. 10.1017/s0962492915000033

[7] Dodge J, Gururangan S, Card D, Schwartz R, Smith NA (2019) Show your work: improved reporting of experimental results. EMNLP-IJCNLP 2019:2185-2194. 10.18653/v1/d19-1224

[8] Duraisamy K (2026) Predictivity and utility of neural surrogates of multiscale PDEs. arXiv:2604.20061.

[9] Franco NR, Manzoni A, Zunino P (2023) A deep learning approach to reduced order modelling of parameter dependent partial differential equations. Math Comp 92(340):483-524. 10.1090/mcom/3781

[10] Fresca S, Manzoni A (2022) POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. Comput Methods Appl Mech Eng 388:114181. 10.1016/j.cma.2021.114181

[11] Greif C, Urban K (2019) Decay of the Kolmogorov N-width for wave problems. Appl Math Lett 96:216-222. 10.1016/j.aml.2019.05.013

[12] Grossmann TG, Komorowska UJ, Latz J, Schönlieb C-B (2024) Can physics-informed neural networks beat the finite element method? IMA J Appl Math 89(1):143-174. 10.1093/imamat/hxae011

[13] Guo M, Hesthaven JS (2018) Reduced order modeling for nonlinear structural analysis using Gaussian process regression. Comput Methods Appl Mech Eng 341:807-826. 10.1016/j.cma.2018.07.017

[14] Hesthaven JS, Ubbiali S (2018) Non-intrusive reduced order modeling of nonlinear problems using neural networks. J Comput Phys 363:55-78. 10.1016/j.jcp.2018.02.037

[15] Huang DZ, Stuart AM (2026) Cost-accuracy trade-offs: neural operator vs classical numerical solver. arXiv:2609.24021.

[16] Kovachki N, Li Z, Liu B, Azizzadenesheli K, Bhattacharya K, Stuart A, Anandkumar A (2023) Neural operator: learning maps between function spaces with applications to PDEs. J Mach Learn Res 24(89):1-97. JMLR 24(89)

[17] Lanthaler S (2023) Operator learning with PCA-Net: upper and lower complexity bounds. arXiv:2303.16317.

[18] Lee K, Carlberg KT (2020) Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. J Comput Phys 404:108973. 10.1016/j.jcp.2019.108973

[19] Li Z, Kovachki N, Azizzadenesheli K, et al. (2021) Fourier neural operator for parametric partial differential equations. ICLR 2021. arXiv:2010.08895

[20] Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE (2021) Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nat Mach Intell 3:218-229. 10.1038/s42256-021-00302-5

[21] Lu L, Meng X, Cai S, et al. (2022) A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. Comput Methods Appl Mech Eng 393:114778. 10.1016/j.cma.2022.114778

[22] McGreivy N, Hakim A (2024) Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. Nat Mach Intell 6(10):1256-1269. 10.1038/s42256-024-00897-5

[23] Mora C, Yousefpour A, Hosseinmardi S, Owhadi H, Bostanabad R (2025) Operator learning with Gaussian processes. Comput Methods Appl Mech Eng 434:117581. 10.1016/j.cma.2024.117581

[24] Rasmussen CE, Williams CKI (2006) Gaussian processes for machine learning. MIT Press. 10.7551/mitpress/3206.001.0001

[25] Westermann J, Huber B, O'Leary-Roseberry T (2026) Performance of neural and polynomial operator surrogates. arXiv:2604.00689.

[26] Williams B, Cremaschi S (2019) Surrogate model selection for design space approximation and surrogate-based optimization. Comput Aided Chem Eng 47:353-359. 10.1016/B978-0-12-818597-1.50056-4

[27] Zhang Y, Roberts N, Marwah T (2026) Breakeven complexity: a new perspective on neural partial differential equation solvers. arXiv:2605.15399.

[28] de Hoop MV, Huang DZ, Qian E, Stuart AM (2022) The cost-accuracy trade-off in operator learning with neural networks. J Mach Learn 1(3):299-341. 10.4208/jml.220509